

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	A. Sreenath	Reg. No.:	19222119
Section / Batch:	B.Tech AIML	Date:	

Code of Conduct 19222119

I, A. Sreenath (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Sreenath

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	3 / 4	4 / 4	90
2	4 / 4	3 / 4	3 / 4	3 / 4	4 / 4	83.75
3	3 / 4	4 / 4	4 / 4	4 / 4	3 / 4	91.25
4	4 / 4	4 / 4	3 / 4	3 / 4	4 / 4	88.75
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	88.75
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N.D.A.

Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

`int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }`

Questions:

- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) $(\epsilon + aa^*)^*$ b) $(a^* + b^*)^* abb$ c) $(a^*b^*)^* aba (a^* + b^*)^*$ d) $(a+ab)^*$ e) $(aba)^* + (a^*b^*)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.

Consider a lexical analyzer for the following tokens:

1. Keywords: if, else, ...
2. Operators: +, -, *, ...
3. Separators: ;, {, }, ...
4. Identifiers: ...

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

① Lexical analysis

== == == ==

lexical analysis is the first phase of the compiler. it reads the source program character by character and converts it into tokens. Tokens are the basic building blocks of a program.

Token Generated.

== == == ==

lexeme	token
p	identifier
=	Assignment operator
(	left parenthesis
a	identifier
*	multiplication operator
b	identifier
)	Right parenthesis
+	Addition operator
(	left parenthesis
b	identifier
*	multiplication operator
c	identifier.
)	Right parenthesis

code optimization

$t_1 = a * b$

$t_2 = b * c$

$t_2 = t_2 * 2$

$p = t_1 + t_2$

target code generator :-

mov R₁, a

mul R₁, b

mov R₂, b

mul R₂, c

SHL R₂, 1

ADD R₁, R₂

mov p, R₁

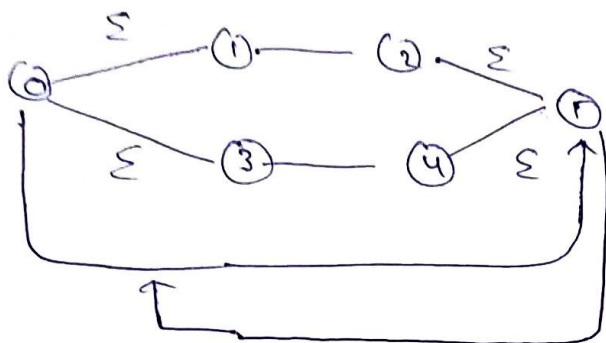
conclusion:-

The expression $p = (a * b) + (b * c) * 2$ passes through all compiler phases through all compiler phases lexical Analysis, Syntax analysis, intermediate code generation, code optimization and target code generation.

2)

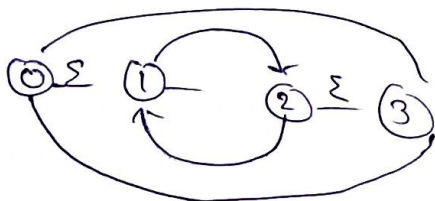
a) $(a+b)^+$

$=$



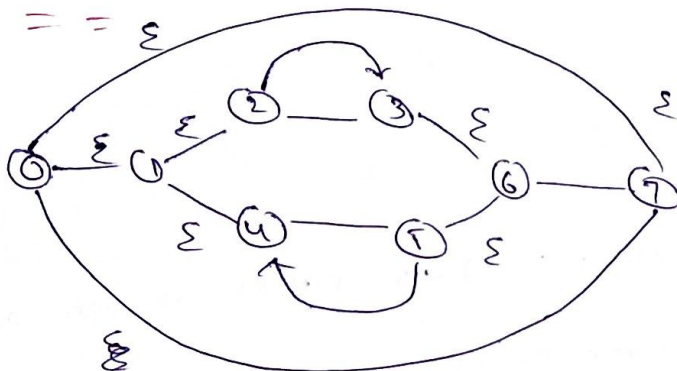
b) ab^*

$=$



c) $(ab^+)^+$

$=$



d) a^*b



total number of tokens

$$= 3 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$$

$$= 30 \text{ Tokens}$$

(4)

even number of 0's

Regular expression:

$$1^* (01^* 01^*)^*$$

explanation:

- 1^* $\rightarrow$ Any number of 1's
- $01^* 01^*$ $\rightarrow$ Two 0's
- $*$ $\rightarrow$ Repeat any number of times

(ii) String containing at least one 1

Regular expression

$$(0+1)^* 1 (0+1)^*$$

explanation.

$(0+1)^*$ $\rightarrow$ Any combination of 0's and 1's

1 $\rightarrow$ At least one 1 must appear.

$(0+1)^*$ $\rightarrow$ Any symbol after that

Regular expression

- keyword - if | else | while | return | int | float
- identifier - $[A-Za-z] + [0-9]^*$
- integer - $[0-9]^+$
- float - $[0-9]^+ \cdot [0-9]^+$
- operator $\rightarrow + [0-9]^+ \cdot [0-9]^+$
- operator - $+ | - | * | / | = | < | > |$
- Separator - $() \{ \} ; ,$

Lexical analysis Algorithm

1. Read the input program
2. ignore spaces, tabs, and new lines
3. check whether the word is a keyword
4. if not a keyword, check whether it is an identifier.
5. check whether for integer or floating - point numbers.
6. identify operators and separators
7. Generate the corresponding token
8. Repeat until the end of the file

example:

```
int var123 = 45;  
float num1 = 23.5;  
if (var123 > 10)  
{  
    return var123;  
}
```

output token

lexeme	token
int	keyword
var123	identifier
=	operator
45	integral literal
;	separator
float	keyword
num1	identifier
23.5	float literal
if	keyword
(	separator
>	operator
10	integral literal
{	separator
{	separator
return	keyword
}	separator

both the number of 0's and 1's are odd

regular expression

$$(00+11)^* (01+10) (00+11)^+$$

explanation:

$(00+11)^*$ $\rightarrow$ add 2 pairs of identical symbols

$-(01+10) \rightarrow$ make both 0's and 1's odd

$-(00+11) \rightarrow$ Allows more even pairs

(iv) String that start with 1 and end with 0

Regular expression:

$$1(0+1)^*0$$

expression explanation
== == ==

start with 1

end with 0

$(0+1)^*$ allows any symbol in between

v) strings that do not contain consecutive 1's

$$(0+10)^* (\epsilon+1)$$

$+10$ ensures every 1 is followed by 0.

-0 can appear anywhere

$-(\epsilon+1)$ allows the string to end with nothing or a single 1

(a+b) ab

= = =



③ definition:-

= = =

A token is the smallest individual unit of a program recognized by the compiler. the five types of tokens are:

- * key words
- * identifiers
- * constants
- * string
- * operators and separators

Token explanation

= = = = =

statement

main c)

{

int a = 10;

char b = "abc";

int c = 30;

char d = "xyz";

int /* comment */ m = 40.5;

tokens

main, (,) = 3

{ = 1

int, a, =, 10, ; = 5

char, b, =, "abc", , ; = 7

int, b, =, 30, ; = 5

char, d, =, "xyz", , ; = 7

int, m, =, 40.5, ; = 5

*

multiplication operator

2

constant

Token Stream:

= = = =

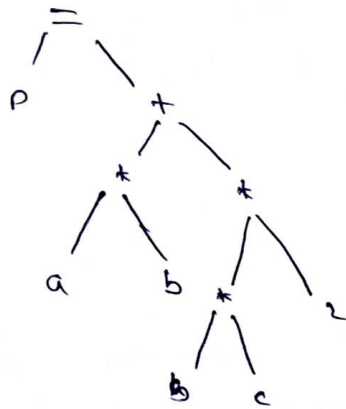
<ID, P> <C> <ID, a> <*> <ID, b> <C>

<+> <C> <ID, b> <*> <ID, c> <C> <*>

<NUM, 2>

Syntax Analysis

= = = =



intermediate code generation.

= = = =

t1 = a * b

t2 = b * c

t3 = t2 * 2

t4 = t1 + t3

P = t4